APPSEC-01: Fundamentals and the Three Pillars of Application Security

> **Series:** APPSEC — Application Security | **Notebook:** 1 of 10 | **Created:** June 2026 | **Last Updated:** 10/28/2026

Overview

Dynatrace Application Security is the platform's security data plane in Gen3 SaaS. It rests on three product pillars — Runtime Vulnerability Analytics (RVA), Runtime Application Protection (RAP), and Security Posture Management (SPM) — and on the same Grail foundation that backs observability data. Findings are first-class entities that flow through workflows, dashboards, and Davis CoPilot like any other Dynatrace signal.

This notebook orients you to the surface area: what each pillar does, what data plane backs it, what OneAgent deployment mode you need, and how the Dynatrace Security Score (DSS) ranks individual vulnerabilities.

Audience: Security engineer, platform owner, AppDev lead onboarding to Dynatrace AppSec.

Outcome: A mental map of Dynatrace AppSec — and a starting place for any AppSec initiative.

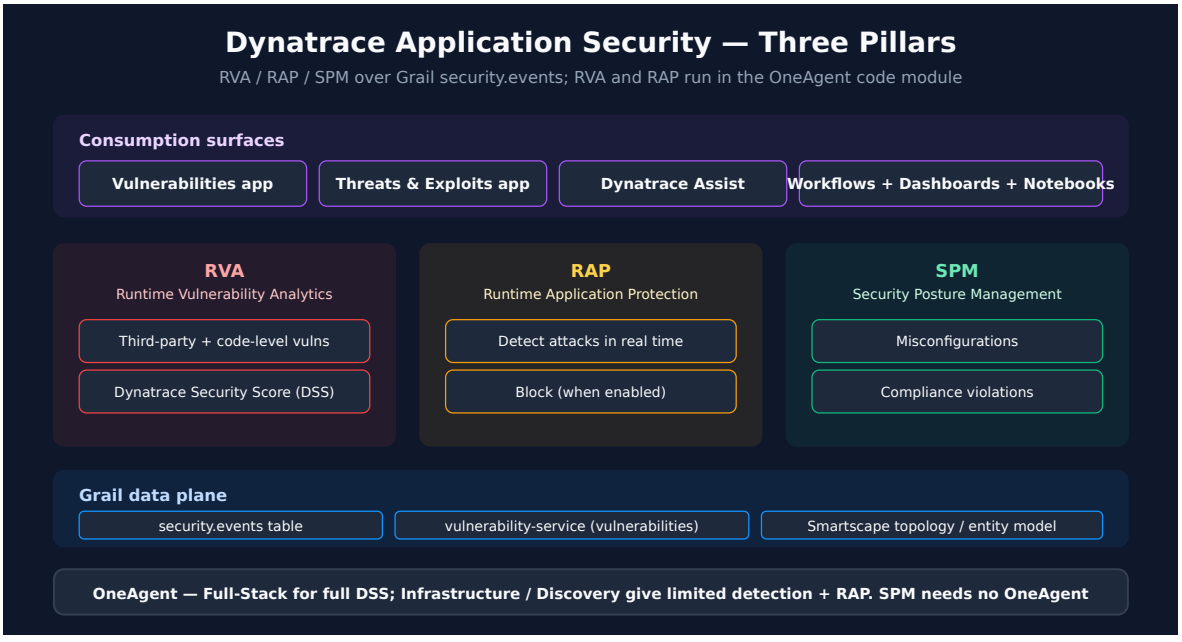


Table of Contents

1. The Three Pillars
2. The Data Plane: Grail and the Vulnerability Service
3. Deployment-Mode Dependency
4. Dynatrace Security Score (DSS)
5. The Series Map
6. Next Steps
7. References

Prerequisites

Requirement	Details
Dynatrace Environment	Gen3 SaaS with Grail (Managed not supported for this series)
OneAgent	Full-Stack monitoring recommended for the full assessment; Infrastructure and Discovery modes give limited vulnerability detection plus RAP, but no environmental DSS adjustment (Discovery also needs code-module injection) — see § 3
AppSec entitlement	Application Security must be enabled on the tenant (consumption-billed in DPS)
DQL familiarity	This series assumes working DQL knowledge. New to DQL? See the SPANS and ORGNZ series first
IAM	To run the queries: `storage:security.events:read` **plus** `storage:buckets:read` (a table permission alone reads nothing) — see APPSEC-09 for the full permission catalog

1. The Three Pillars

The three pillars are separate DPS capabilities, each enabled and billed on its own: Runtime Vulnerability Analytics and Runtime Application Protection in GiB-hours, Kubernetes Security

Posture Management in host-hours (each node scanned during an hour counts as one host-hour). RVA and RAP run in the OneAgent code module; SPM is independent of OneAgent. All three write their findings to the same Grail `security.events` table and share the entity model.

Runtime Vulnerability Analytics (RVA)

Continuously inspects the running process inventory and answers two questions: *which CVEs are reachable from production code paths*, and *which production data assets are exposed to those code paths*. RVA covers both third-party (library) vulnerabilities and code-level (first-party) vulnerabilities where supported.

Distinguishing feature vs scanner-based tools: RVA reports a third-party vulnerability only for components a running process has actually loaded, not for every declared dependency. For Java, the optional *Java vulnerable function reporting* OneAgent feature also shows whether the vulnerable function is in use. That is a triage filter; it does not change DSS (§ 4).

Runtime Application Protection (RAP)

Sits in the OneAgent code module and watches request traffic for four documented attack classes: SQL injection and command injection on Java, .NET and Go, and JNDI injection and SSRF on Java only. Each technology's attack control is **Off**, **Monitor** (detect only), or **Block**; custom monitoring rules scoped to process groups or vulnerability types override that global control. Start in Monitor and promote to Block once detection has been tuned (APPSEC-04).

RAP needs deep monitoring of the process — the RAP docs list it as a prerequisite — so processes without deep monitoring produce no detections.

Security Posture Management (SPM)

Evaluates the configuration state of Kubernetes clusters, cloud accounts (AWS, Azure, GCP), and VMware environments against compliance standards (CIS, PCI DSS, NIST, DORA, and others) and surfaces the results as findings in the same `security.events` table as RVA/RAP events.

Where RVA/RAP are *runtime* signals (what's executing right now), SPM is a *config* signal (what's been declared in the platform).

> **Sources:** Application Security (DT docs) for the three-pillar framing and RVA/RAP/SPM definitions; Runtime Application Protection (DT docs) — *"Detection of SQL injection, JNDI injection, command injection, and SSRF attacks"* and *"For Runtime Application Protection to work properly, make sure deep monitoring is enabled"* (its technology table checks JNDI injection and SSRF for Java only); Vulnerability

> Analytics (DT docs) — *"Issues a vulnerability only when the component is in use."*; Vulnerabilities concepts (DT docs) — vulnerable functions are *"not considered for the DSS calculation"*; DPS capability pages for RVA, RAP and SPM — *"The unit of measure for Kubernetes Security Posture Management is a host-hour."*; Security Posture Management (DT docs) — *"Security Posture Management provides comprehensive visibility into the security posture of your Kubernetes, cloud, and VMware environments."* (all re-read 10/02/2026). **Derived:** the *runtime vs config* distinction is a synthesis aid; the hub page does not phrase it this way directly.

2. The Data Plane: Grail and the Vulnerability Service

AppSec produces two kinds of records that you'll query, alert on, and dashboard:

Surface	Where it lives	How to read it	How to manage it
Security events (RVA state and change events, RAP detection findings, SPM compliance findings)	Grail `security.events` table	DQL: `fetch security.events`	Read-only — triage happens in the Vulnerabilities and Threats & Exploits apps; IAM: `storage:security.events:read` + `storage:buckets:read`
Vulnerabilities (deduplicated across affected entities)	`vulnerability-service` (not a Grail bucket)	**Vulnerabilities** app; API; Davis CoPilot	Mute affected entities, add ticket links (app / API); IAM: `vulnerability-service:vulnerabilities:read` / `:write` + the app's default policies (APPSEC-09)

This split matters for two reasons:

1. **DQL works on events, not problems.** Custom dashboards built on DQL pull from `security.events`. You can't `fetch security.problems` — that surface is exposed via the vulnerability-service API and the Vulnerabilities app. (The classic *Security Problems* / *Third-Party Vulnerabilities* apps are deprecated.)
2. **IAM splits across Grail and the vulnerability-service.** DQL needs `storage:security.events:read` plus `storage:buckets:read`; the Vulnerabilities app and API need `vulnerability-service:vulnerabilities:read`. The `environment:roles:*security-problems` roles serve only the deprecated classic apps.

There is no single `storage:security_problems:read` token — see APPSEC-09 for the complete model.

A first DQL to run against your tenant to see what's actually flowing:

```
// All security events in the last 7 days grouped by event.type
// Use this to discover which event types your tenant is producing
fetch security.events, from:-7d
| summarize count = count(), by:{event.type}
| sort count desc
```

If that query returns rows, AppSec is producing data. If it returns zero rows, either AppSec is not enabled on the tenant, no vulnerabilities have been detected yet, the OneAgent code module is not attached to your monitored processes, SPM scans have not run in the window, or you lack `storage:buckets:read`. Widen the window to `from:-30d` before concluding.

> **Sources:** Application Security (DT docs), IAM policy statements reference (DT docs) for permission-token names verified verbatim. **Softened:** the absence of a `storage:security_problems:read` token reflects the IAM reference as of 10/02/2026; verify in your tenant's policy editor before relying on this gap to inform a policy design.

3. Deployment-Mode Dependency

AppSec coverage is not a license toggle alone — it depends on **how OneAgent is deployed on each host**. The difference between modes is less about *whether* findings appear and more about *how well they are assessed*.

OneAgent mode	Third-party vulnerabilities	Code-level vulnerabilities	RAP	DSS environmental adjustment	SPM
Full-Stack	✓	✓	✓	✓	independent of mode
Infrastructure	limited	limited	✓	✗ — DSS equals the CVSS base score	independent of mode

OneAgent mode	Third-party vulnerabilities	Code-level vulnerabilities	RAP	DSS environmental adjustment	SPM
Discovery	limited (needs code-module injection)	limited (needs code-module injection)	✓ (needs code-module injection)	✗ — DSS equals the CVSS base score	independent of mode

Practical implication: a tenant where most hosts run in Infrastructure or Discovery mode still produces vulnerability findings and RAP detections, but their risk ranking is flat. Without Full-Stack topology, Dynatrace cannot tell whether a vulnerable process is exposed to the public internet or reaches sensitive data assets — those values show as *Not available* — so every finding keeps its CVSS base score. The failure mode is mis-prioritization, not silence: a CVE on an internal batch host ranks the same as one on an internet-facing payment service.

Recommended rollout order: put business-critical, internet-facing workloads on Full-Stack first (or enable code-module injection where Discovery mode is used), let RVA accumulate a baseline (typically 24–72 hours), *then* start prioritizing on DSS. Reading DSS on a fleet that is mostly Infrastructure mode reads CVSS with a different label.

> **Sources:** Application Security (DT docs) — the *Monitoring modes coverage* table (third-party and code-level detection *limited* in Infrastructure and Discovery; RAP in all three), *"In an Infrastructure Monitoring deployment, Dynatrace Intelligence cannot adapt the Dynatrace Security Score"*, *"the DSS will be the same as the CVSS base score"*, *"Infrastructure Monitoring mode lacks environmental information, such as reachable data assets or public internet exposure"*, *"For Application Security to work in Discovery mode, after enabling Discovery mode, you also need to enable code-module injection."*, and *"Dynatrace Security Posture Management (SPM) works independently of monitoring modes."* (re-read 09/18/2026). **Softened:** the 24–72 hour baseline is community-practice guidance — verify against your environment's vulnerability cadence.

4. Dynatrace Security Score (DSS)

DSS is a **per-vulnerability** risk score (0.1–10.0). For third-party vulnerabilities it starts from the CVSS base score, and environmental context from your topology can only lower or keep it — DSS never exceeds the CVSS base score. Code-level vulnerabilities are always 10 / Critical. It is not a tenant-wide posture number: every vulnerability carries its own DSS. In Grail the score is `vulnerability.davis_assessment.score` and its level is `vulnerability.risk.level` (the

semantic dictionary still describes the score as the *Davis Security Score*). A higher DSS means a more severe vulnerability.

Environmental context that adjusts the score:

- **Public internet exposure** — a vulnerability on an internal-only process is scored lower than the same vulnerability on a process reachable from the public internet. One caveat: on Linux hosts where topology cannot supply this, exposure is detected via eBPF instead, and those eBPF-derived states (*Public network* / *Not detected*) do **not** influence DSS.
- **Reachable data assets** — does an affected process have database access (run a database service)?

Where that context is missing — hosts in Infrastructure or Discovery mode (§ 3) — no adjustment happens and DSS equals the CVSS base score.

What's not published: the exact weighting formula.

How to consume DSS in practice:

- Use the per-vulnerability DSS to **prioritize** — work the CRITICAL and HIGH levels first.
- To **trend posture**, count open vulnerabilities per DSS level and chart the counts over time (the APPSEC-02 § 3 query, run on a schedule). Rising CRITICAL/HIGH counts are deterioration; falling counts are progress.
- Pair those counts with the backlog burn-rate from APPSEC-08: a rising count with a flat close rate means remediation is not keeping up.
- Don't set absolute count thresholds across business units without normalizing for estate size and monitoring-mode coverage — a Full-Stack estate surfaces more findings adjusted *down* from CVSS than an Infrastructure-mode one, whose findings keep their CVSS base score.

> **Sources:** Vulnerabilities concepts (DT docs) — *"DSS never exceeds the original CVSS Base Score; environmental modifiers can only reduce or maintain the score."*, *"The DSS scale ranges between 0.1 (lowest risk) and 10.0 (most critical risk)"*, *"The score of a code-level vulnerability is always 10 and the risk always Critical"*, and reachable data assets means *"at least one process that has database access (runs a database service)"*; Application Security (DT docs) for the environmental signals and *"public internet exposure is detected via eBPF. Potential states are Public network and Not detected. Dynatrace Security Score isn't influenced by either of these states."* (re-read 10/02/2026). **Dictionary:** `vulnerability.davis_assessment.score` (`stable`), `vulnerability.risk.level` (`stable`), read 09/18/2026. **Softened:** the cross-unit normalization guidance is community practice.

5. The Series Map

Where to go next depending on what you're trying to do.

#	Notebook	When to read
02	Runtime Vulnerability Analytics	You need to triage third-party vulnerabilities by exposure and vulnerable-function use
03	Code-Level Vulnerability Analytics	You're responsible for first-party application code and need to action findings by entry point and code location
04	Runtime Application Protection	You're setting up RAP detection rules and deciding when to promote to blocking
05	Security Posture Management	You're auditing configuration drift against CIS/PCI/NIST baselines
06	Kubernetes & Container Security	You're running workloads under DynaKube and need image + cluster posture together
07	Investigations & Dynatrace Assist for Security	You're conducting an investigation and want AI-assisted queries
08	Workflows, Notifications & Remediation	You need vulnerabilities and attacks to reach Jira / ServiceNow / PagerDuty / Slack
09	IAM and Gen3 Permissions for AppSec	You're designing the permission model — who can see what, who can manage what
10	Dashboards, Reporting & Governance	You're building the executive view and governance cadence

If you're net-new to AppSec on this tenant: read 01 → 09 (IAM) → 02 (RVA) → 04 (RAP) → 10 (dashboards). IAM comes early because rolling out AppSec to users without a permission plan creates either over-exposure of sensitive payloads or under-exposure that blocks the SOC.

6. Next Steps

1. **Verify AppSec is producing data** — run the `fetch security.events` query above. If it returns zero rows, work the enablement and deployment-mode questions before continuing.
2. **Map your OneAgent coverage** — for each business-critical workload, confirm Full-Stack mode; Infrastructure and Discovery hosts still report findings, but without the environmental

DSS adjustment (§ 3).

3. **Read APPSEC-09 next** — get the permission model right before granting the SOC access to vulnerabilities. Mistakes here are reversible but visible.
4. **Then read APPSEC-02** — third-party vulnerabilities are usually the loudest finding type at first, so triaging RVA is where most AppSec rollouts start producing measurable signal.

7. References

Source	Coverage
Application Security (DT docs)	Three-pillar hub page; DSS signals; monitoring-modes coverage table
Vulnerabilities concepts (DT docs)	DSS as a per-vulnerability score built on CVSS
Runtime Application Protection (DT docs)	RAP attack classes, supported technologies, attack control
Security Posture Management (DT docs)	SPM flavors (KSPM / CSPM / VSPM) and compliance standards
Secure (DT docs)	Top-level Secure navigation
Secure / FAQ (DT docs)	Official AppSec FAQ
IAM policy statements reference (DT docs)	Verified AppSec permission tokens

⚠️ **DISCLAIMER**: This information was AI generated and is provided "as-is" without warranty. It was produced as an independent, community-driven project and **not supported by Dynatrace**. Always refer to official Dynatrace documentation for the most current information.